Data and Model Versioning

Introduction

Data and Model Versioning

Overview

Agenda

- [08:15 – 09:00] – Lecture: General data versioning strategies
- [09:15 – 10:00] – Lecture: Continued
- [10:15 – 11:00] – Exercises: DVC
- [11:15 – 12:00] – Exercises: Continued

Preparation

For the exercises:

- DVC: Install
- DVC: Getting started
- DVC: Tutorial

For the lecture:

- DVC
- ML Engineering, Chapter 3.11 (http://bit.ly/MLEbook-Chapter3)

- ML-Ops.org: Data Engineering Pipelines

Notes

- Make sure to install DVC and test that the `dvc` command works for you.

Lecture Intro

Data and Model Versioning

Outline

The lecture covers the following topics:

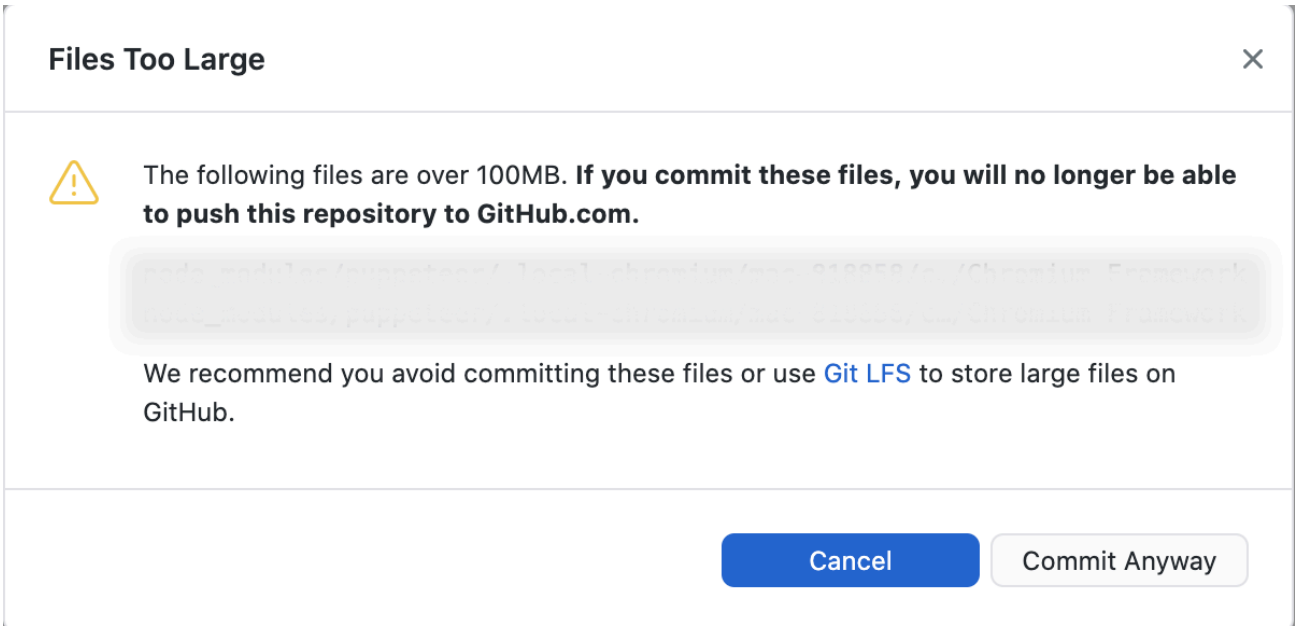
1. Why data versioning?
2. Data storage
3. Data stages (medallion)
4. Data version operations
5. Metadata for versions
6. Data stages in ML



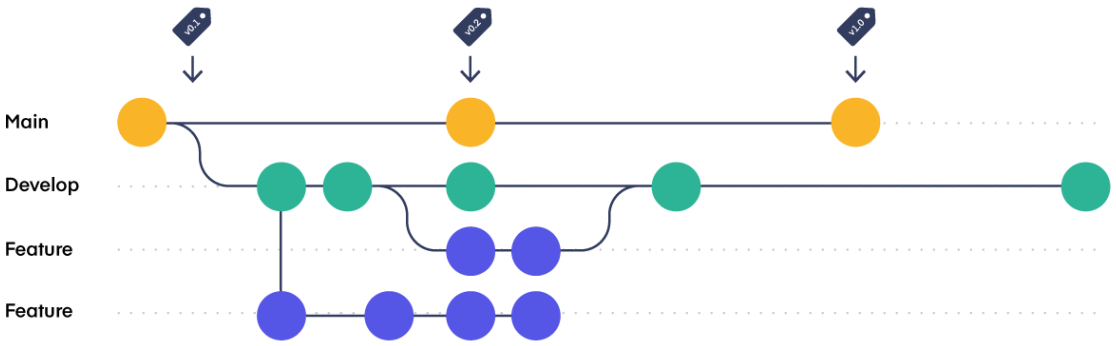


Motivation

Why Data Versioning?



Unique reference to data



From https://www.pablogonzalez.io/salesforce-git-branching-strategies/

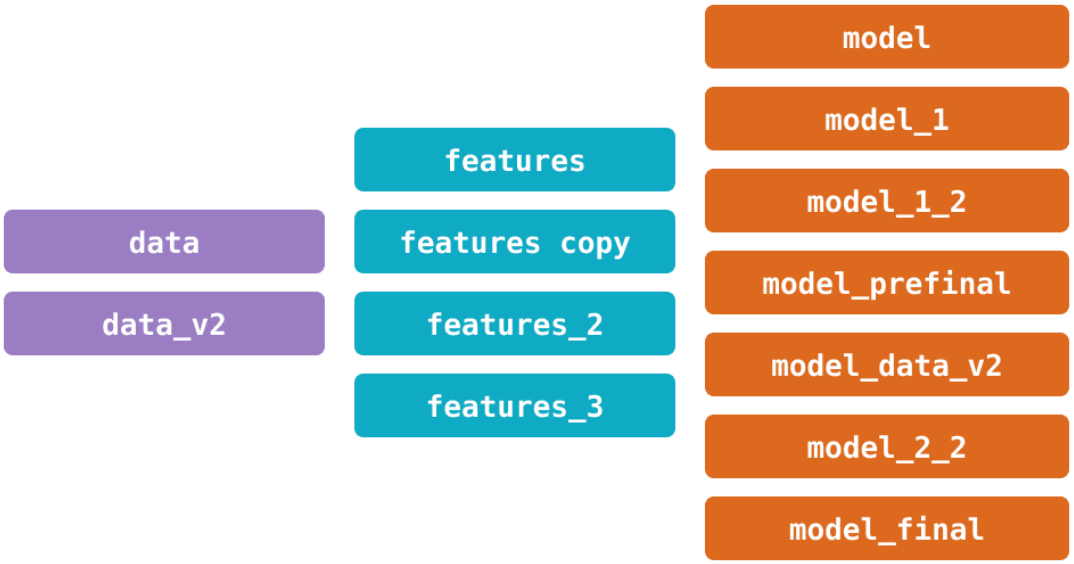
Like with git and code:

- **Unique reference**: Each version of your data has a unique identifier
- **Versioned changes**: Track changes to your data over time

- **Revert/rollback**: Ability to revert to previous versions when needed
- **Check previous performance**: Reproduce and verify model performance at any point in

Complexity of Data Science Projects

Complexity of Data Science Projects

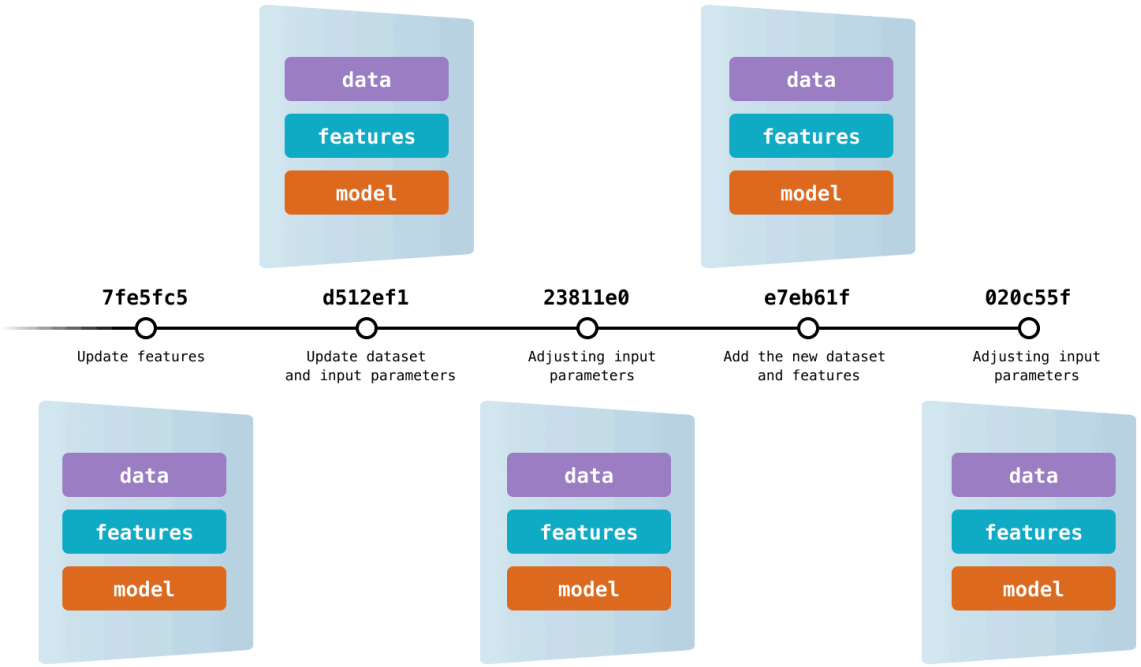


From https://dvc.org/doc/use-cases/versioning-data-and-models

Familiar?

Happens without planning. Not production viable.

Packaged solutions



From https://dvc.org/doc/use-cases/versioning-data-and-models

- One commit, one "package
  - Data
  - Features
  - Models/artifacts
- Applies to all kinds of data
  - Images
  - Tabular
  - etc

Levels of Data Versioning

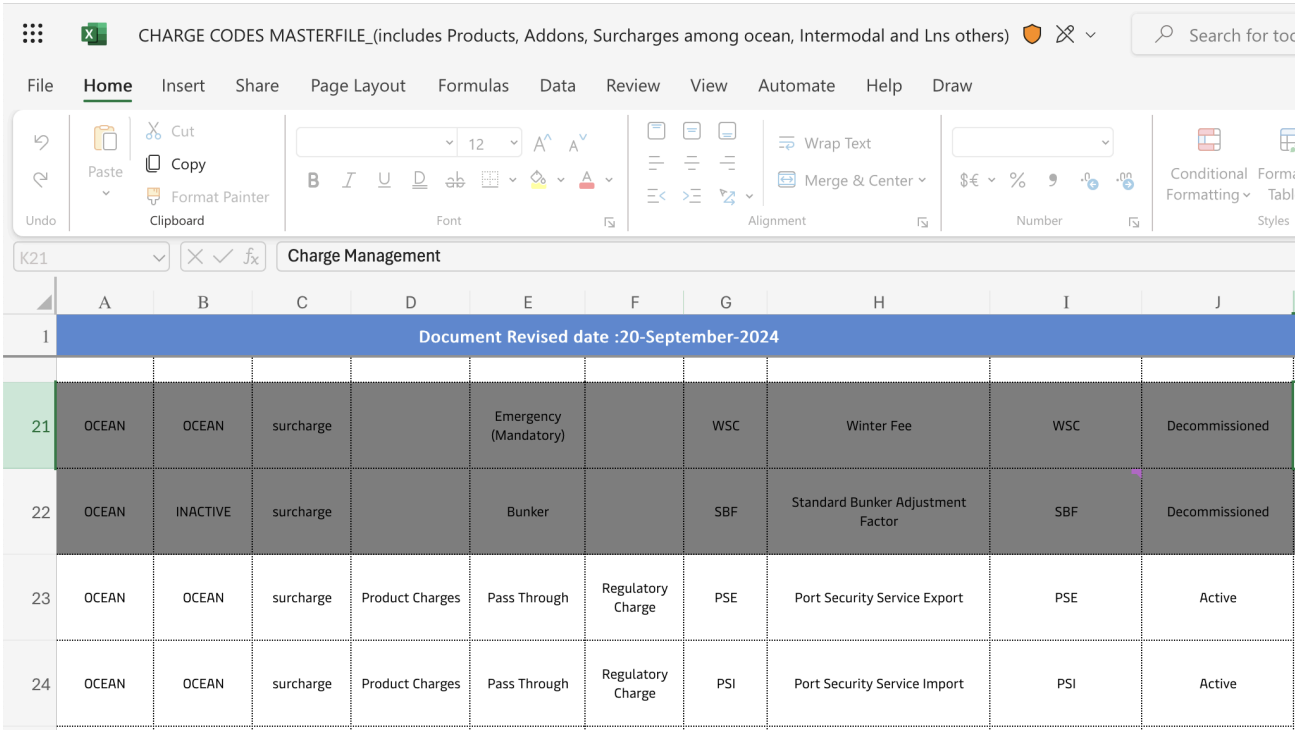
Multiple Levels of Data Versioning

4 levels according to the MLE book, chapter 3.11

Level 0	Level 1	Level 2	Level 3
Data is unversioned	Data is versioned as a snapshot	Data and code are versioned	Data and code are versioned through specialised tools

Level 0: Unversioned Data

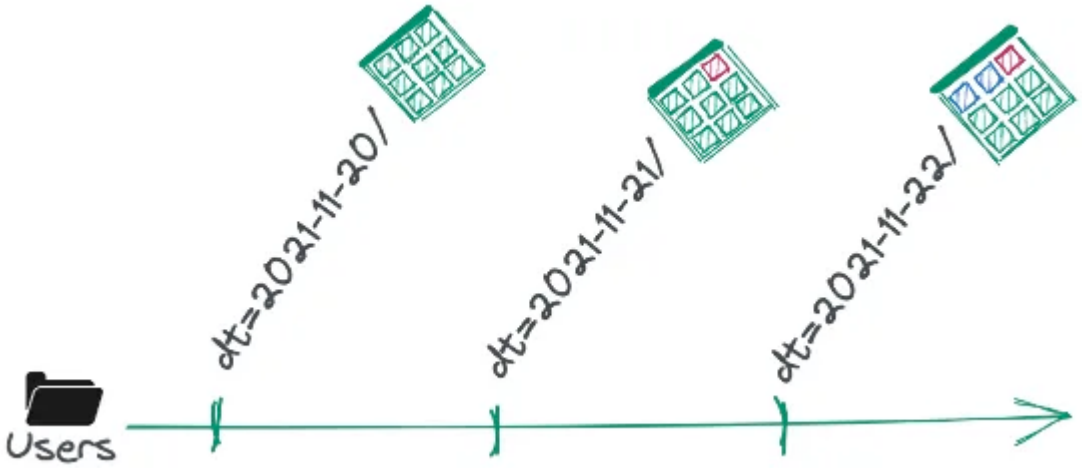
The simplest (and least recommended) approach is to have no versioning at all. Data is simply stored and overwritten as needed.



- Simple to implement
  - No overhead
- No history
  - Not possible to revert
- Note: many tools have *something* built in

Level 1: Snapshots

Create snapshots of your data at different points in time. This allows you to restore previous versions.

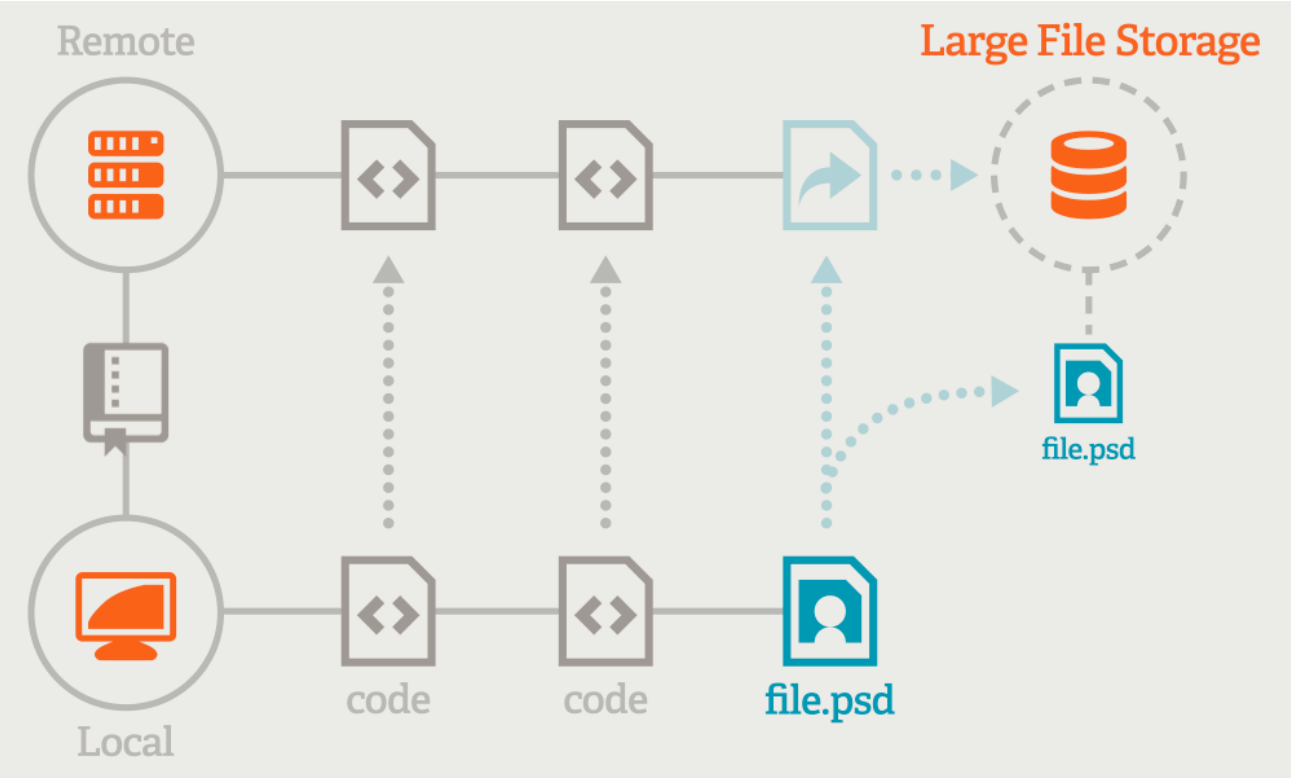


From https://lakefs.io/blog/data-versioning/

- Duplicate all relevant data
  - Storage intensive → cloud
  - Manual tracking of versions
- Viable if:
  - Infrequent updates
  - Few models

Level 2: Data and Code as One Asset

Version data alongside code using Git with large file support.

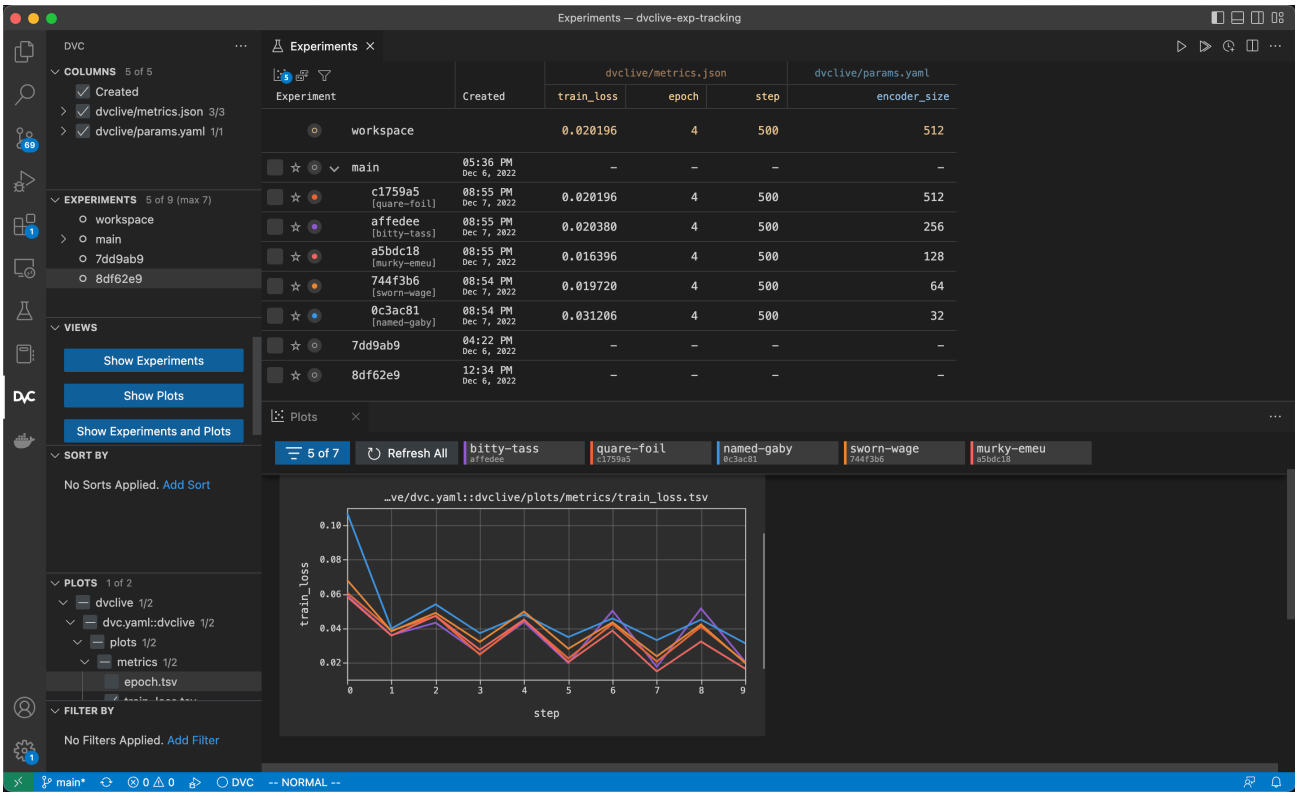


From https://git-lfs.com/

- Version control with Git
  - Metadata saved for large files
- Large files stored in cloud
  - Efficient formats
- Tools:
  - DVC
  - Git Large Files Storage

Level 3: Specialized Versioning

Use specialized tools designed for data versioning that handle large files efficiently.



From https://dvc.org/doc/use-cases/experiment-tracking

- Tool-specific solutions
- Comes with a lot extra functionality
    - Pipelines
    - Experiment tracking
    - Etc etc

Data Version Operations

Data Version Operations

Standard Flow

The standard workflow for versioning data with DVC follows these steps:

0. Initialize DVC
1. Add data files to DVC tracking
2. Add DVC metadata files to git
3. Commit the changes to git
4. Push the changes to git
5. Push data to remote storage

Demo Time!

0. Initialize DVC

```
dvc init
```

1. Track files with DVC

```
dvc add data.csv
```

This creates a `.dvc` file that contains metadata about the data file.

2. Git add

```
git add data.csv.dvc
```

3. Git commit

```
git commit -m "Add data.csv to DVC tracking"
```

4. Push to Remote

```
dvc push
```

This uploads the actual data to your configured remote storage.

Info in .dvc Files

When you run `dvc add`, a `.dvc` file is created with the following information:

Demo example

```
outs:
- md5: a1b2c3d4e5f6...
  size: 1048576
  nfiles: 1
  path: data.csv
```

Metadata explanation:

- `path`: The relative path to the data file
- `hash/md5`: The MD5 (or other) hash algorithm/value of the file content
- `size`: Size of the file in bytes

- `nfiles`: Number of files (for directories)

Project example

```
md5: 4feb90a977597b6b3f3b34dc2d4d3711
frozen: true
deps:
- checksum: '"0fbbe2410507589f580fd0a63f0b10f8"'
  size: 114686
  hash: md5
  path: https://itudsip.hel1.your-objectstorage.com/data/images/metadata.csv
outs:
- md5: 0fbbe2410507589f580fd0a63f0b10f8
  size: 114686
  hash: md5
  path: metadata.csv
```

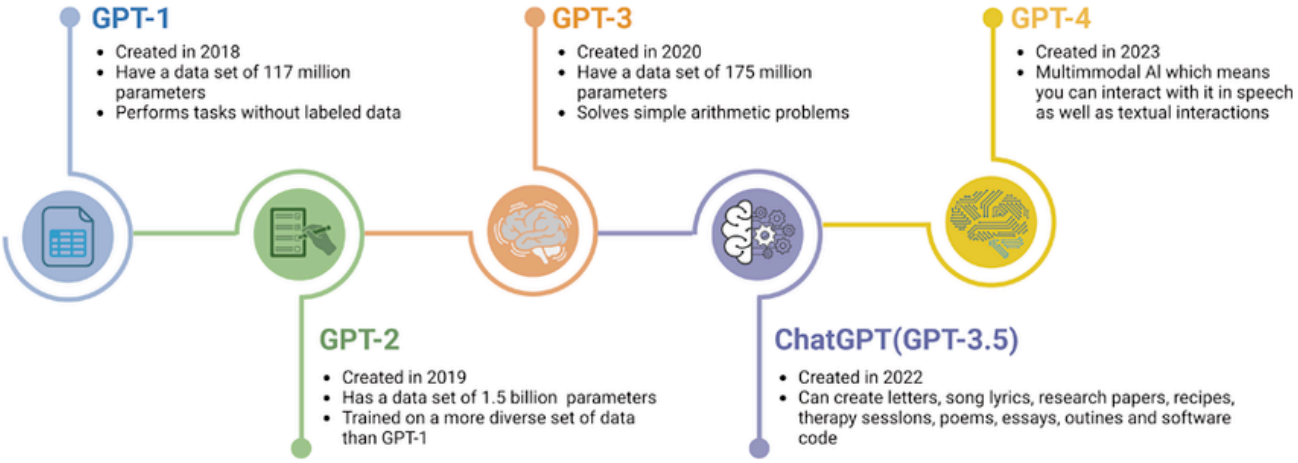
Metadata explanation:

- `md5`: Checksum of the DVC file itself (identifies this specific version of the pipeline stage)
- `frozen: true`: The stage is locked and will not be re-executed even if its dependencies change, until explicitly unlocked with `dvc unlock`
- `deps`: Dependency entries. Only present when dvc import or dvc import-url are used to generate this .dvc file
    - `checksum`: Hash of the dependency file content
    - `size`: Size of the file in bytes
    - `hash`: Hash algorithm used (md5, sha256, etc.)
    - `path`: Path to the dependency (relative to wdir, which defaults to the file's location)
- `outs` (outputs): Files produced by this stage
    - `md5`: Hash of the output file content
    - `size`: Size of the file in bytes
    - `hash`: Hash algorithm used

- `path`: Path to the file or directory relative to working directory

---

Release Tags

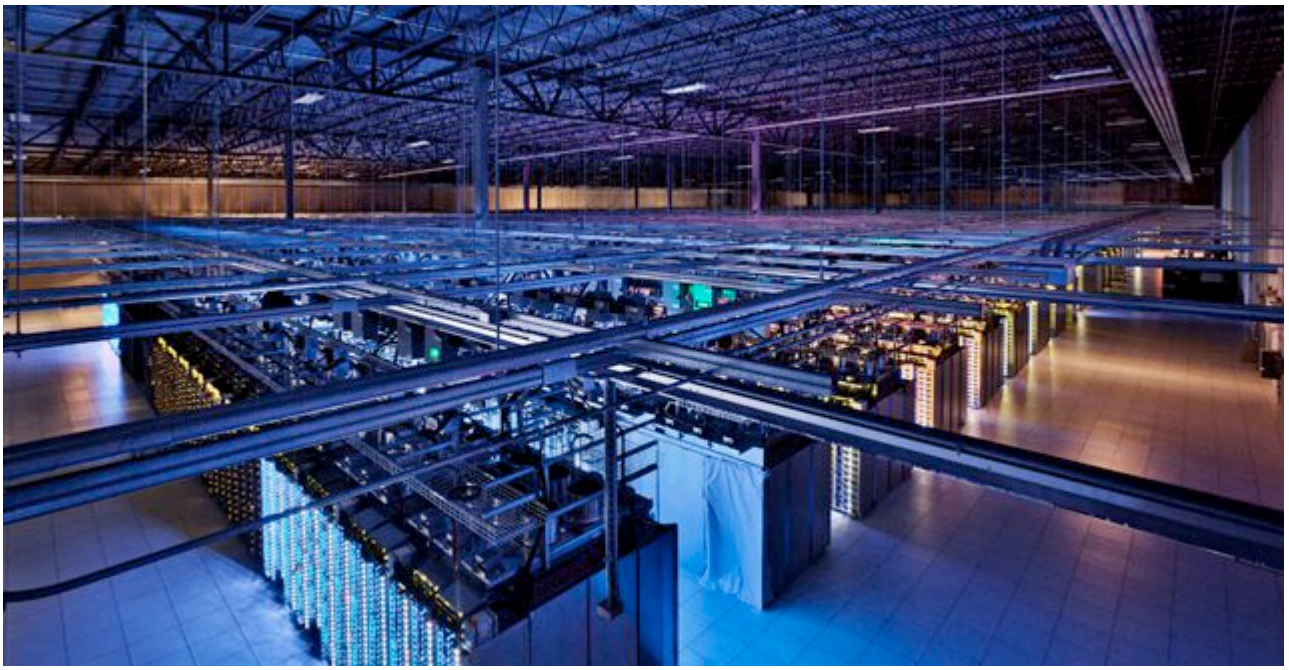


From https://www.researchgate.net/figure/Illustration-of-GPT-development-history-and-the-rise-of-ChatGPT-The-development-timeline_fig1_374092520

- Do it through git
  - `git tag -a "v2.0" -m "imagenet v2.0"`
- Examples:
  - GPT
  - Do you have any?

Storage Cost

Data Storage Cost



From https://lpigroup.com/portfolio-item/data-centre-netherlands/

Storage Costs Money

Guesses for big cloud providers?

Answers:

Provider	Hot / Standard	Infrequent Access / Cool	Cold	Archive	1TB Hot Storage Cost
AWS S3	$0.023/GB (first 50 TB)	$0.0125/GB (S3 Standard-IA)	$0.004/GB (Glacier Instant) · $0.0036/GB (Flexible)	$0.00099/GB (Glacier Deep Archive)	$23.00
Azure Blob	$0.018/GB (Hot, LRS)	$0.010/GB (Cool)	$0.0045/GB (Cold)	$0.00099/GB (Archive)	$18.00
Google Cloud Storage	$0.020/GB (Standard, regional)	$0.010/GB (Nearline, regional)	$0.004/GB (Coldline)	$0.0024/GB (Archive, US/EU multi-region)	$20.00
Oracle OCI	$0.0255/GB (Object Standard)	$0.015/GB (Infrequent Access)	—	$0.0026/GB (Archive)	$25.50

From https://www.finout.io/blog/cloud-storage-pricing-comparison

Cost Strategies

Cloud vs On-Premise

On-Premise Storage:

- Capital expenditure upfront
- Full control
- Potential security benefits
- Maintenance overhead

Cloud Storage:

- Scalable
- Pay-as-you-go
- Managed by provider
- Easy to access from anywhere

Hot vs Cold Storage

	Cold	Hot
Access Speed	Slow	Fast
Access Frequency	Seldom or Never	Frequent
Data Volume	Low	High
Storage Media	Slower drives, LTO, offline	Faster drives, durable drives, SSDs
Cost	Lower	Higher

From https://www.backblaze.com/blog/whats-the-diff-hot-and-cold-data-storage/

Hot Storage:

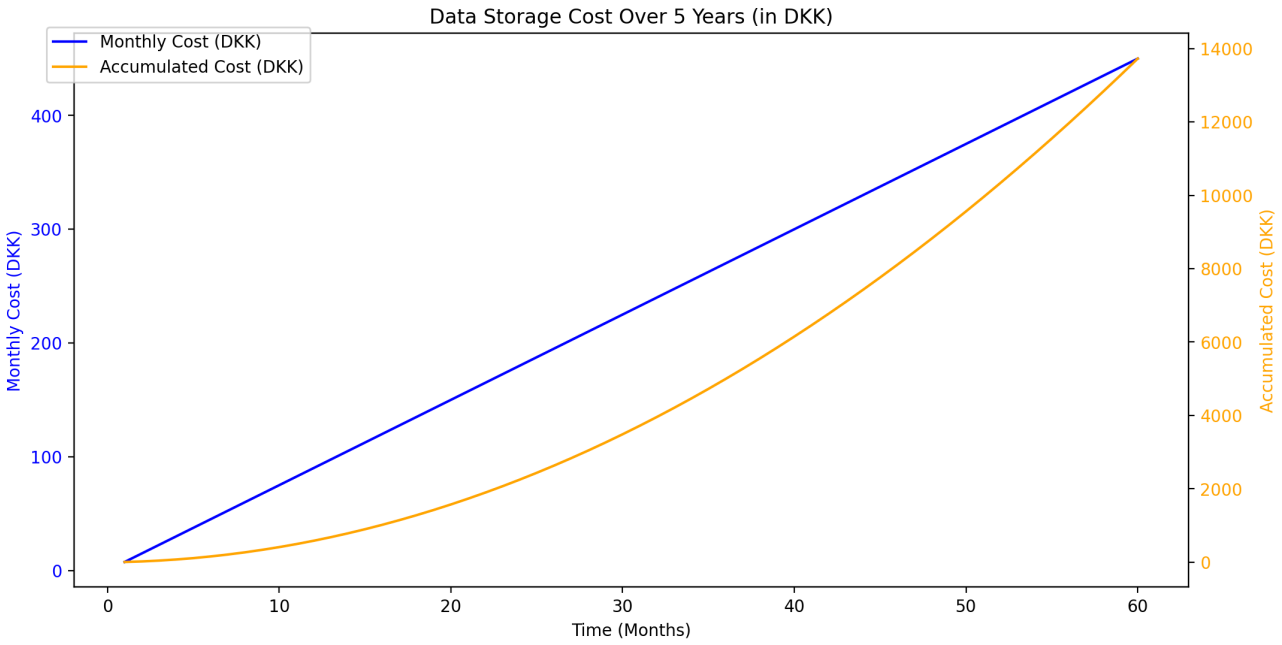
- Frequently accessed data
- Faster access times

- Higher cost per GB
- Examples: SSD, fast HDD, in-memory

Cold Storage:

- Rarely accessed data
- Slower access times
- Lower cost per GB
- Examples: Glacier, archive storage, tape

Retention Policies



How long different types of data should be kept.

- Indefinite is tempting
    - Running costs can explode, though
- Raw data can be kept for longer

    - Processed data can be regenerated

Partitioning

Split data into logical partitions:

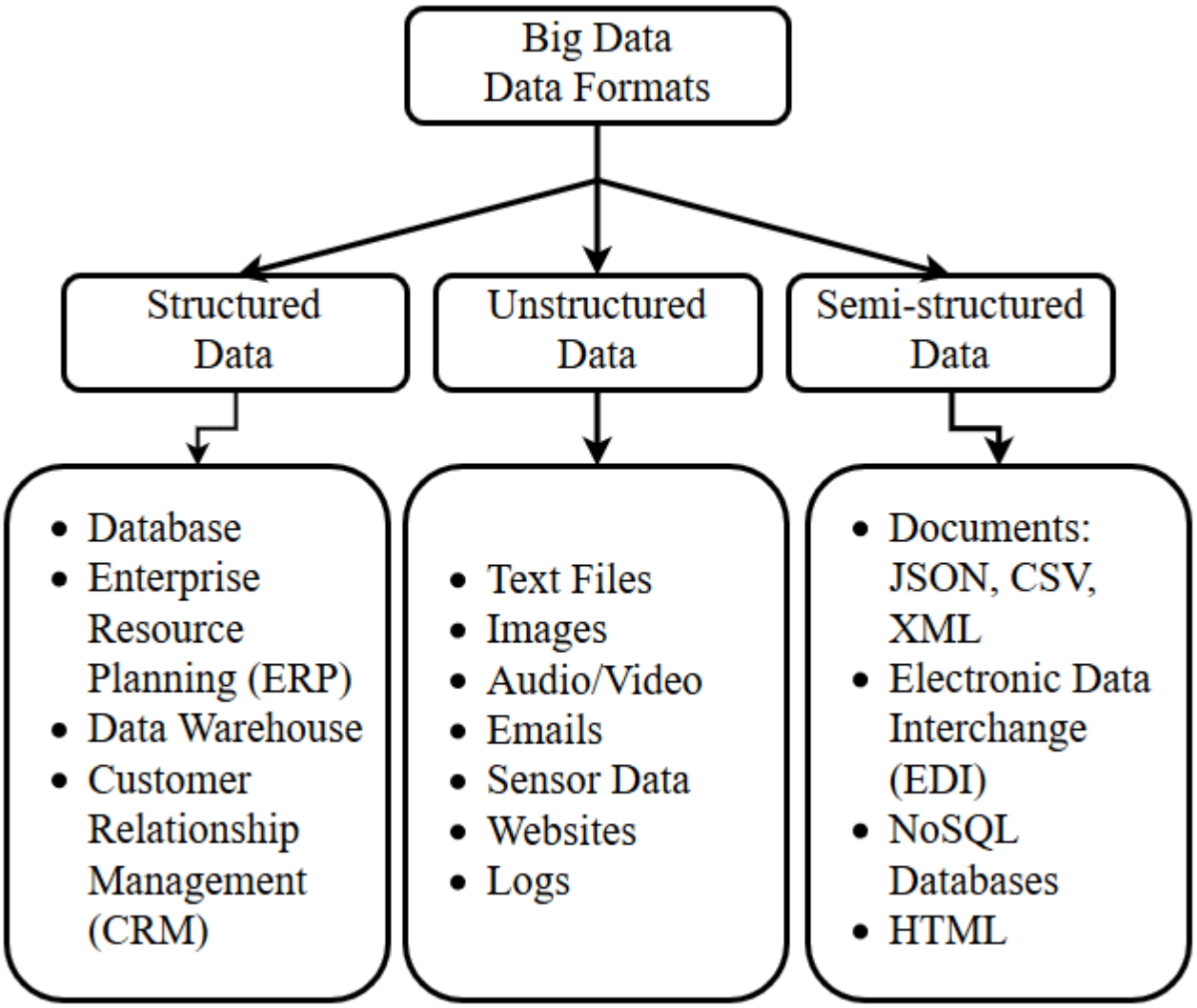
- By date (year/month/day)
- By project
- By data type
- By version

Storage Recommendations

1. **Use appropriate storage tiers**: Hot for active data, cold for archives
2. **Implement lifecycle policies**: Automatically move data between tiers
3. **Compress data**: Use appropriate compression for each data type
4. **Deduplicate**: Avoid storing the same data multiple times
5. **Monitor costs**: Regularly review storage usage and costs

Data Formats

Data Formats



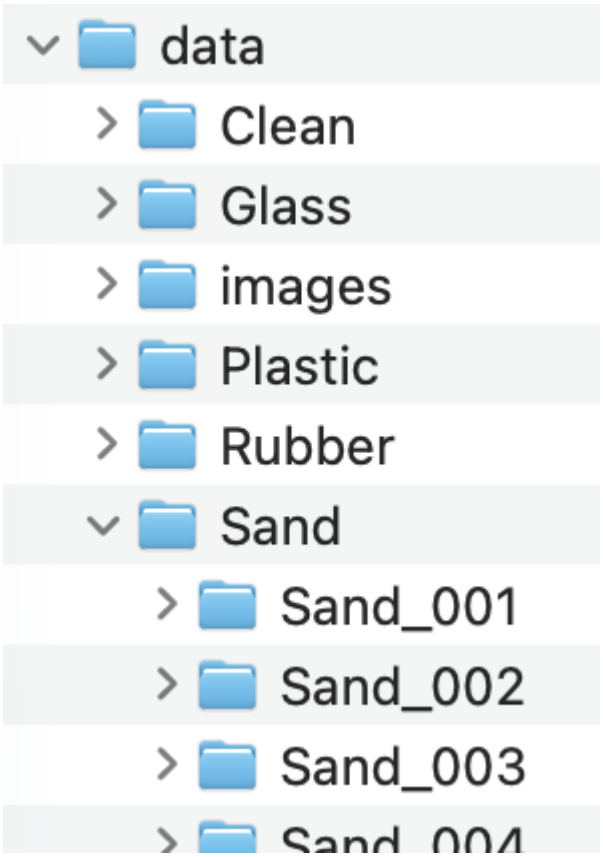
From Testing Big Data Application - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Big-Data-data-formats_fig3_339027369 [accessed 1 Aug 2026]

Structured vs Unstructured Data

- Structured
  - Defined schema
  - Relational, tabular, DB, ...
- Unstructured
  - Variable format

    - Texts, images, ...
- Semi-structured
    - JSON text, ...

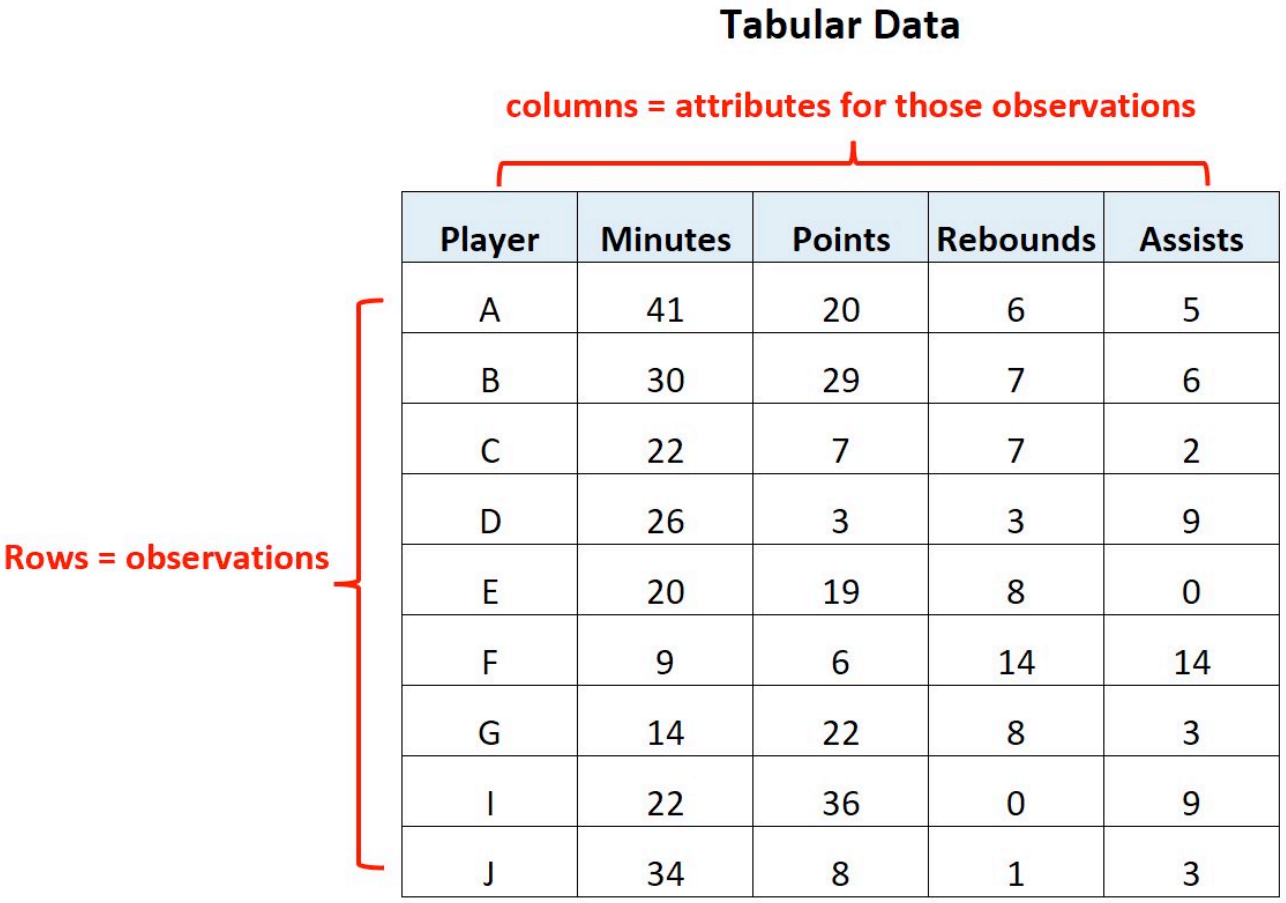
And this?



Common Formats

Tabular/Relational Data

Structured data organized in tables with rows and columns.



From https://www.statology.org/tabular-data/

- One row per ID, organised in tables
- Commonly associated files:
  - CSV
  - Parquet
  - ORC
- Easy to query and analyze

Images (RAW files, compressed)

Image data comes in various formats with different storage requirements.

Image quality	Image size	File size [1]
NEF (RAW), Lossless compressed, 12-bit	—	32.4 MB
NEF (RAW), Lossless compressed, 14-bit	—	41.3 MB
NEF (RAW), Compressed, 12-bit	—	29.0 MB
NEF (RAW), Compressed, 14-bit	—	35.9 MB
NEF (RAW), Uncompressed, 12-bit	—	57.0 MB
NEF (RAW), Uncompressed, 14-bit	—	74.4 MB
TIFF (RGB)	L	108.2 MB
	M	61.5 MB
	S	28.0 MB
JPEG fine [3]	L	16.3 MB
	M	10.4 MB
	S	5.2 MB
JPEG normal [3]	L	9.1 MB
	M	5.3 MB
	S	2.6 MB
JPEG basic [3]	L	4.0 MB
	M	2.7 MB
	S	1.4 MB

From https://mcpactions.com/nikon-raw-s-file-size-option/

- RAW files are much larger than compressed formats
- Compression may lose information
- Different formats for different use cases

How would you store raw and compressed images? Hot and/or cold?

Semi-structured

```
{
    "glossary": {
        "title": "example glossary",
                  "GlossDiv": {
            "title": "S",
                          "GlossList": {
                "GlossEntry": {
                    "ID": "SGML",
                                            "SortAs": "SGML",
                                            "GlossTerm": "Standard Generalized Ma
                                            "Acronym": "SGML",
                                            "Abbrev": "ISO 8879:1986",
                                            "GlossDef": {
                        "para": "A meta-markup language, used to create markup lar
                                                    "GlossSeeAlso": ["GML", "XML
                    },
                                            "GlossSee": "markup"
                }
            }
        }
    }
}
```

- Some structure, no schema
- Commonly used for configurations

Storage Architectures

Data Storage Architectures



Based on https://www.databricks.com/research/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics

- **Lakes**: raw, unfiltered
- **Warehouses**: processed, vetted
- **Lakehouses**: Both?
- **ELT** vs **ETL**
    - Extract, Load/Transform
    - Steps before saving/storing data

Data Warehouses

Optimized for structured data and analytical queries.





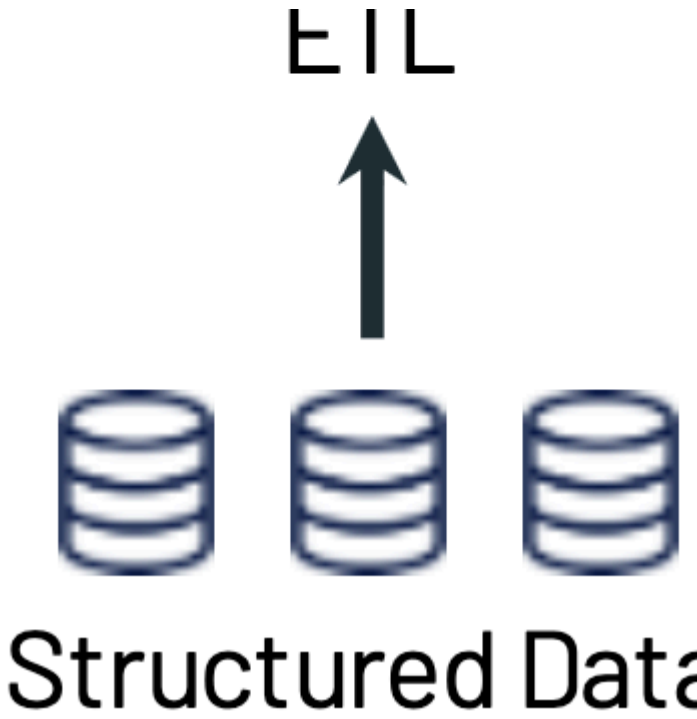
BI Reports



Data Warehouses



ETL



From https://www.databricks.com/research/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics

Characteristics

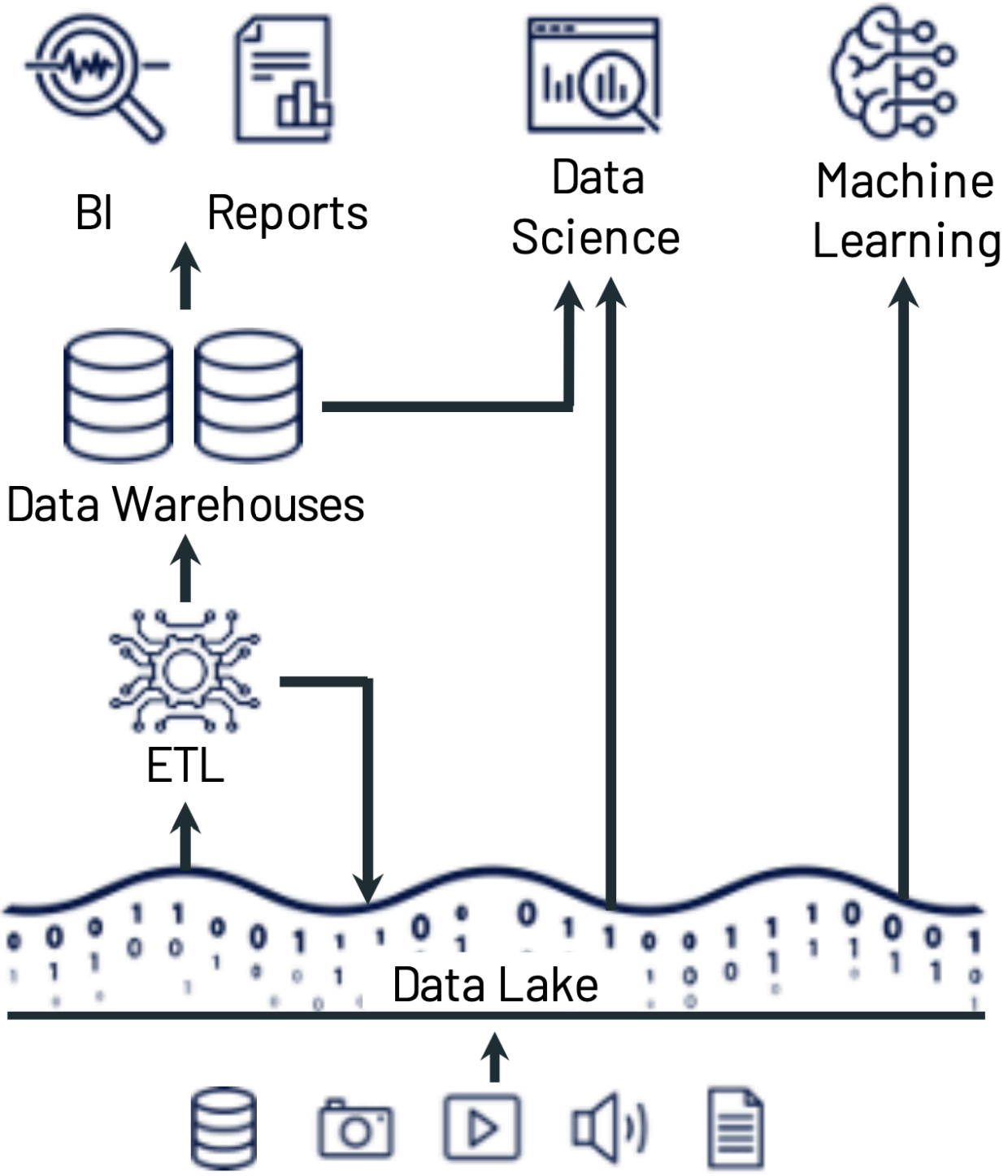
- Schema-on-write
- Optimized for SQL queries
- Good for reporting and BI

Limitations

- Lack of customizability for AI/ML
- Images and unstructured data
- Higher cost for large volumes

Data Lakes

Centralized repositories that hold raw data in its native format, kicked off with Hadoop.



From https://www.databricks.com/research/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics

Characteristics

- Schema-on-read
- Supports various data types
- Cost-effective for large volumes

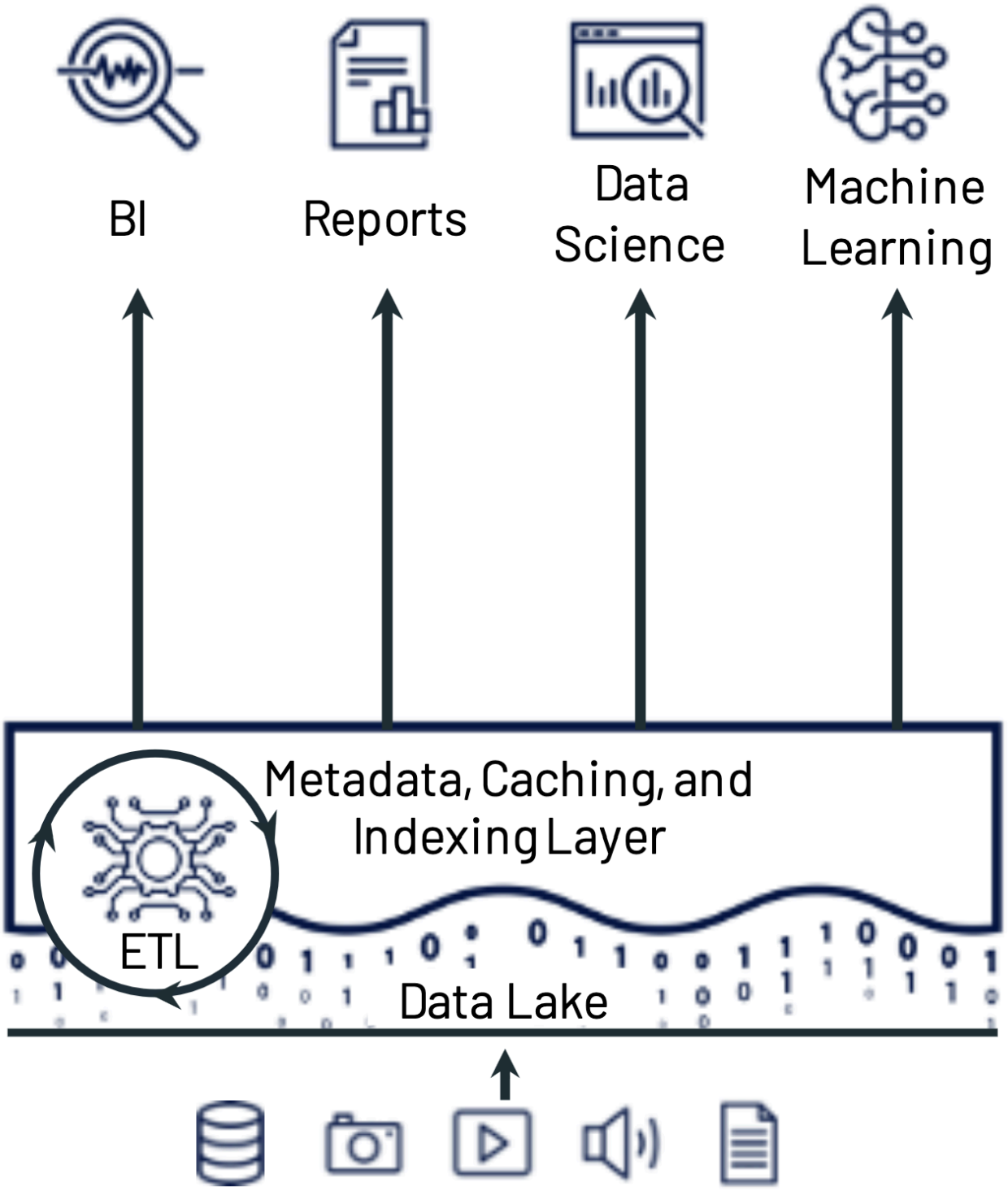
- Good for exploration and discovery

Limitations

- Requires additional, complex ETL loops
- Delayed data for BI/analytics
- Cost of ownership (duplicate data, harder dvc, no ACID transactions)

Data Lakehouses

Combination of data lakes and data warehouses.

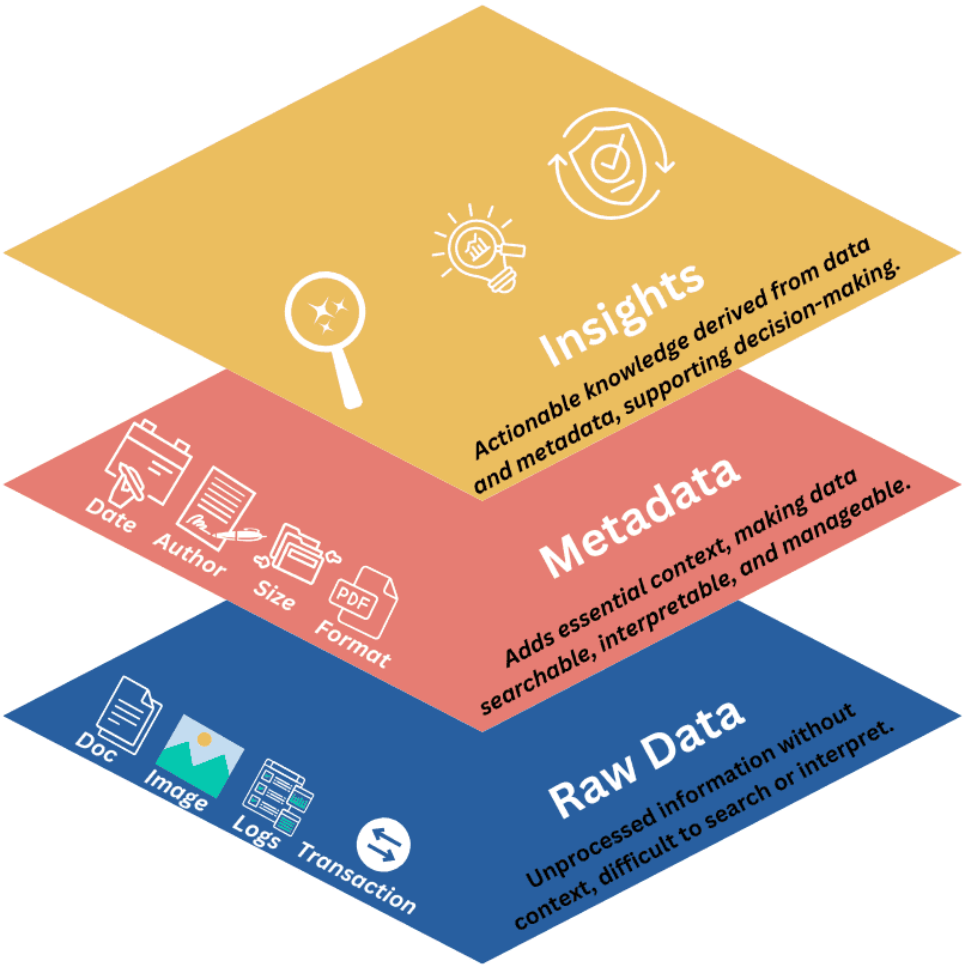


From https://www.databricks.com/research/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics

- ACID transactions on data lake storage
- Support for both structured and unstructured data
- Optimized for both analytics and AI/ML
- Open formats (Parquet, Delta, etc.)

Metadata

Metadata is crucial for understanding and managing data versions.



From https://www.acceldata.io/blog/what-is-metadata-definition-types-and-importance

Descriptive

- **Description**: Human-readable description of the data
- **Keywords**: Tags and subjects for categorization
- **Titles**: Dataset or table names
- **Authors**: Who created the data

Structural

- **Schema**: Data structure, column names, types
- **Partitioning**: Physical layout rules
- **Relationships**: How tables and datasets are connected
- **Statistics**: Min, max, mean, null counts, etc.
- **Data Quality**: Quality scores, validation results

Administrative

- **Author**: Who created or modified the data
- **Timestamp**: When was it created or modified
- **Version**: Version identifier (tag, commit hash, etc.)
- **Changelog**: What changed between versions
- **Ownership**: Who is responsible for the data
- **Policy**: Access management, sensitivity labels, permissions

Technical

- **Format**: File format (Parquet, CSV, JSON, etc.)
- **Size**: Dataset size and dimensions
- **Compression**: Compression method used

- **Resolution**: Data granularity or precision

Provenance

- **Lineage**: Where did this data come from (data lineage)
- **Origin**: Source systems and extraction methods
- **History**: Complete transformation history

Usage

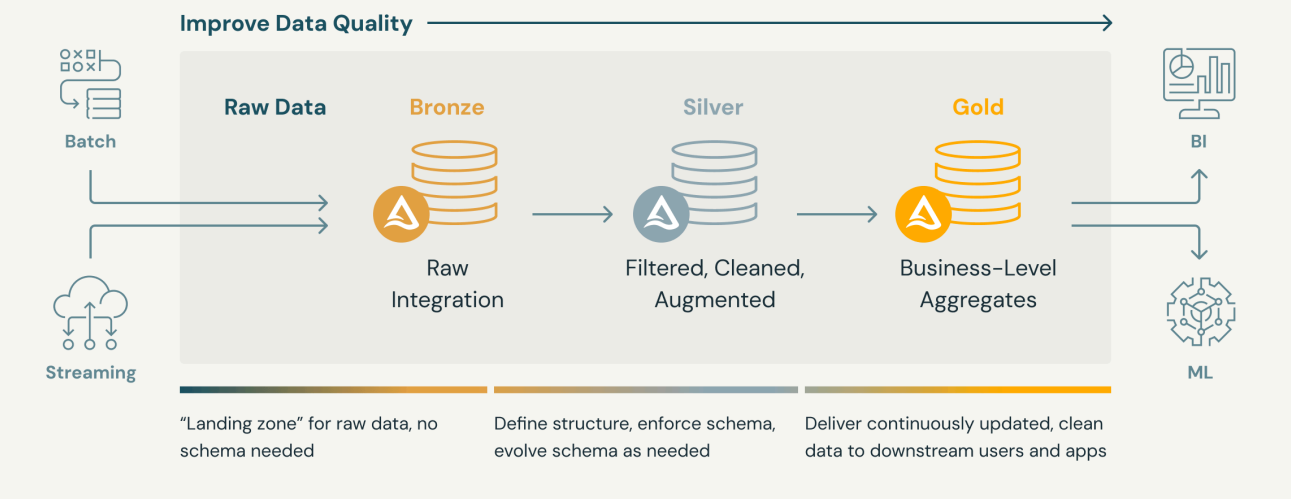
- **Usage**: Monitoring usage and cost estimates
- **Access patterns**: Who accesses the data and when
- **Query history**: Track how data is queried

Data Stages

Data Stages

Medallion Architecture

The medallion architecture, popularized by Databricks, is a data organization pattern that structures data into quality layers.



From https://www.databricks.com/glossary/medallion-architecture

🥉 Bronze Layer

- Raw, unprocessed data
- High volume, high velocity
- Schema may evolve over time
- No or minimal transformations

🥈 Silver Layer

- Data has been cleaned (nulls handled, outliers removed)
- Schema is more stable
- Some business logic applied
- Data scientists typical users

🥇 Gold Layer

- Enriched with business context
- Aggregated and summarized
- Optimized for querying

- Ready for analytics and ML

Version control where?

Version control can be applied at different stages:

- **Bronze**: Version the raw data files themselves
- **Silver**: Version the cleaning/transformation code
- **Gold**: Version the aggregation and business logic

Best practice: Version at all stages to ensure full reproducibility.

ML Datasets



From https://www.rawgeneration.com/products/best-juice-cleanse when searching on Google for "raw cleansed curated".

Raw data

- Unprocessed
- May contain errors

- May have inconsistent formats
- Often needs significant cleaning
- Bronze

Cleansed data

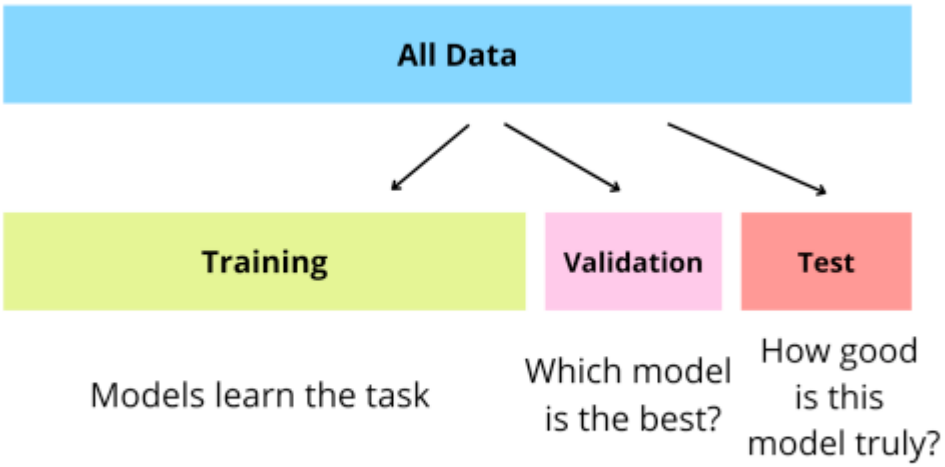
- Missing values handled
- Outliers identified or removed
- Duplicate records removed
- Consistent formats
- Silver

Curated/model-ready data

- Features engineered
- Categorical variables encoded
- Numerical features scaled
- Balanced classes (if applicable)
- Ready for model consumption
- Gold

Other Considerations

Train/validate/test split



From https://www.kaggle.com/discussions/general/516681

Where in the Medallion structure would you do this?

Between bronze and silver.

Controlling randomness

To ensure reproducibility, always set random seeds:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

# Set random seeds
np.random.seed(42)
pd.options.mode.chained_assignment = None  # default='warn'

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
import tensorflow as tf

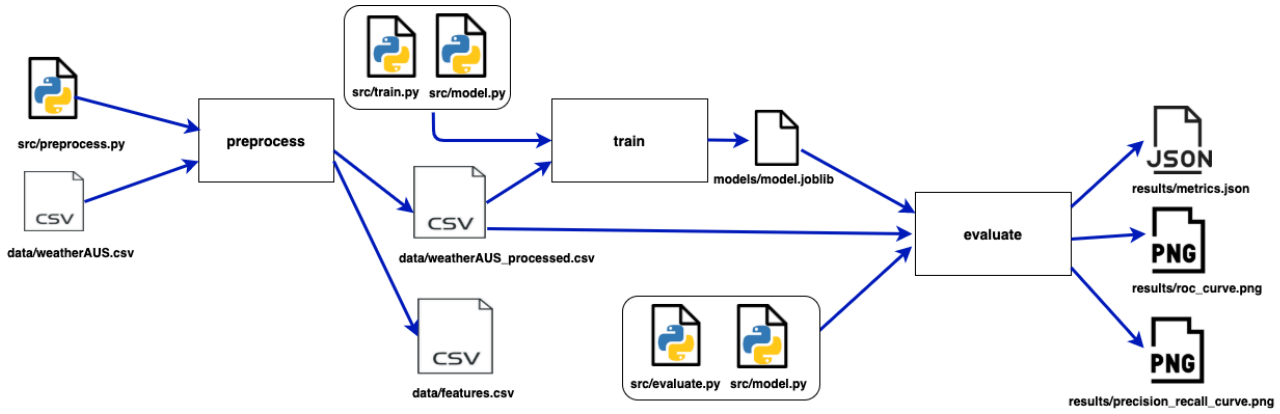
# For TensorFlow
tf.random.set_seed(42)
```

```
import torch

# For PyTorch
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

ML Pipelines

ML data pipelines move data through various processing stages to prepare it for model training.



From https://mlops-guide.github.io/Versionamento/pipelines_dvc/

1. **Preprocessing**: Extract, validate, split, feature engineering, normalization
2. **Training**: Train/run model, extract outputs
3. **Postprocessing**: Format output, evaluate results, monitoring

→ More on this in future lectures

Best Practices

1. **Clear boundaries**: Each stage should have well-defined inputs and outputs
2. **Quality gates**: Validate data quality at each stage
3. **Documentation**: Document the purpose and transformations of each stage
4. **Testing**: Test transformations between stages
5. **DAGs**: Directed acyclic graphs -- watch out for infinite loops

Summary

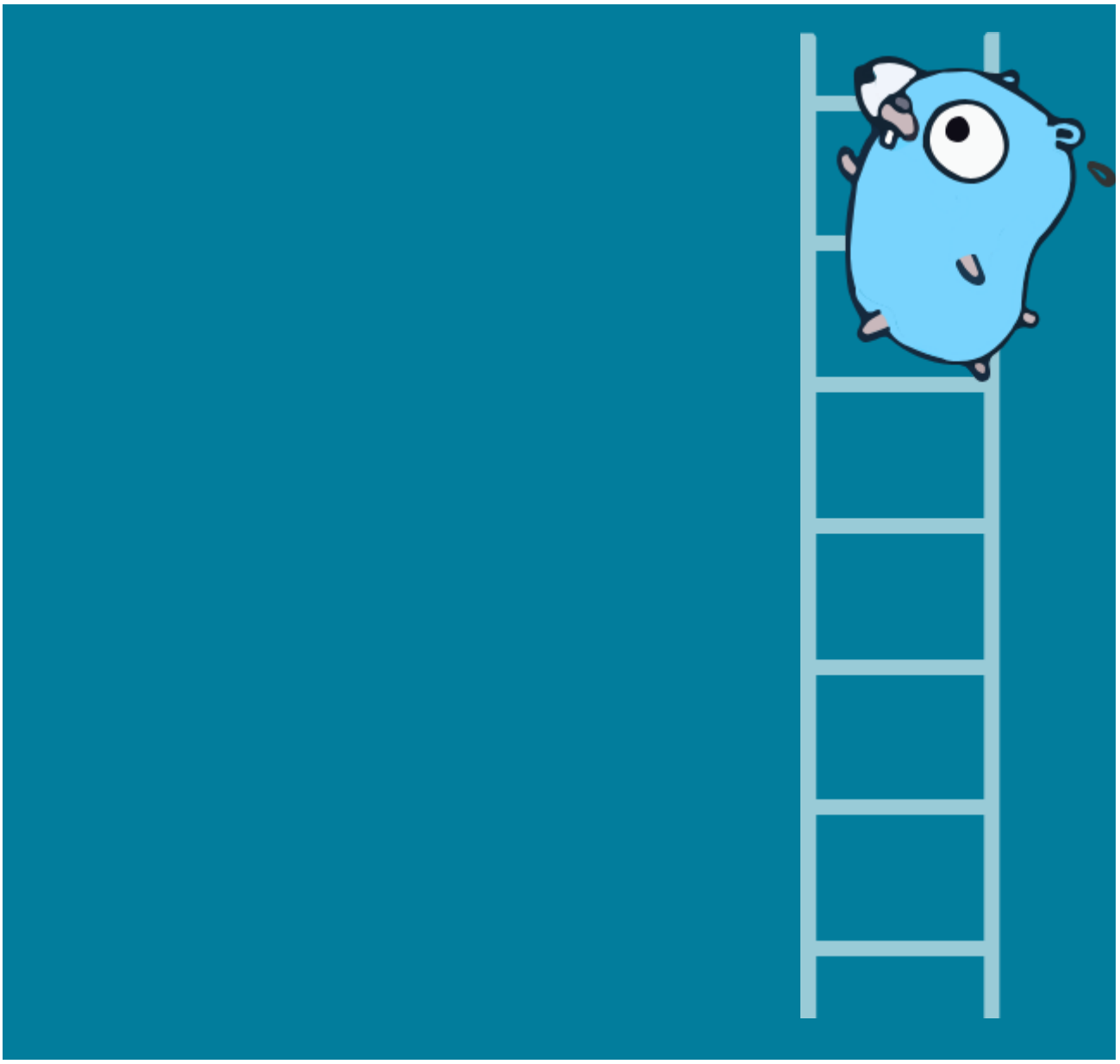
Summary

1. **Data and model versioning**

    - Version code, data, features & model in one package
2. **4 versioning levels**
    - From unversioned to specialized tooling
3. **DVC as a tool**
    - Keep large files remote
4. **Storage considerations**
    - Cost, data formats
    - Data warehouse/lake/lakehouse
    - Metadata
5. **Data stages**
    - Medallion architecture
    - Raw, cleansed, curated

Next lessons

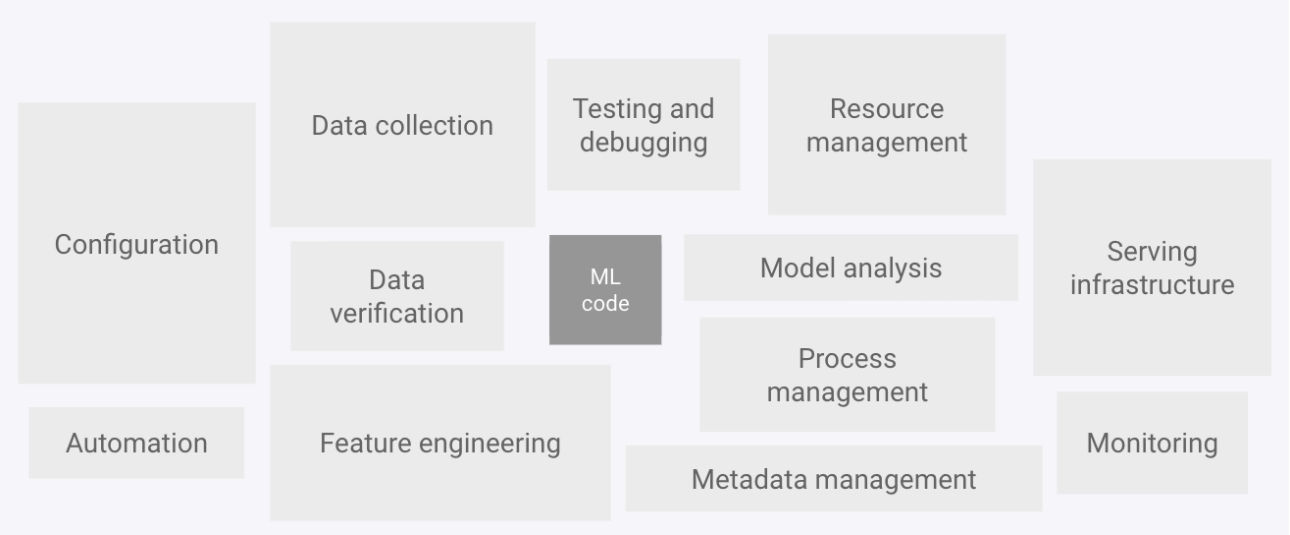
Golang



Fundamentals of Go with Lasse.

MLOps Overview and ML Project Structure

Lecture: MLOps theory and its 3 aspects (data, model, code), and implementations in organisations.



From https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

Exercises: Use project templates in Cookiecutter Data Science to ensure consistency across ML projects.



From https://cookiecutter-data-science.drivendata.org/

Resources

- DVC Documentation
- ML Engineering Book, Chapter 3.11
- ML-Ops.org: Data Engineering Pipelines

- LakeFS Blog
- Git LFS

Exercise Intro

Background

Data versioning has plenty of similarities but also differences to normal version control that you have just worked with.

The main thing that we want to accomplish with data version control is to be able to always go back to one version of the data and see how it looked like at a given time -- similar to normal version control. If you want to evaluate your model's performance at time t_1, it does not make sense to use data from time t_2.

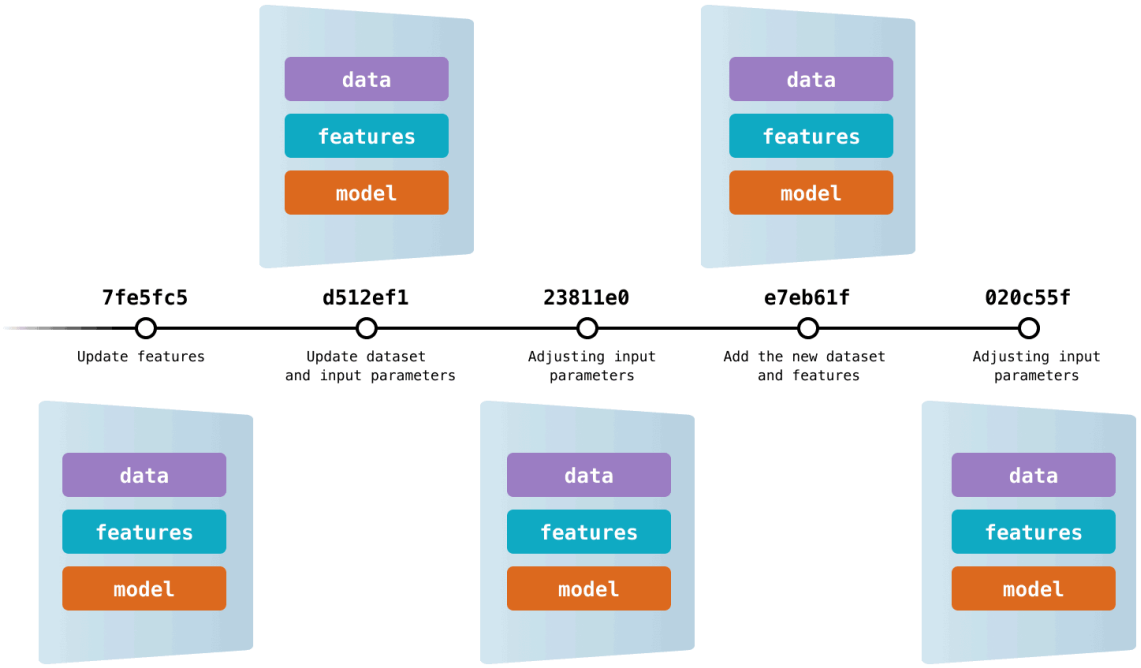
One big difference is that it is simply not possible to meaningfully look at the differences between data versions and review it through a PR. Good PRs tend to only touch a few lines of code at a time, but data can be millions of rows/lines. Not only would it take *you* ages, but it can also greatly slow down any version control software that has to calculate all the differences. And the data can be enormous, so checking out the whole repo would be impossible on your "small" laptop.



So what's a good strategy to keep track of changes to your data? Briefly explained, you store the data in a remote location and version control *references* to the data instead.

For that purpose, we will use the tool *Data Version Control* (DVC). It can also be used for model versioning and experiment tracking, but we'll touch on that in later lectures. Similar tools include Git Large File Storage (GitLFS) or Pachyderm.

Each of these tools have their pros and cons, and DVC might struggle with performance if you have a lot of files, but it serves our purpose of refreshing our git skills and motivate the thinking of storing data remotely well enough.



The goal of the following exercises is to take you through the journey of linking data to your project.

Exercises

The following exercises will guide you through using DVC for data and model versioning:

- Exercise 1: Initialization - Initialize in your repository
- Exercise 2: Start Tracking Files - Add files to DVC tracking
- Exercise 3: Using a Remote - Configure and use remote storage
- Exercise 4: Switching Between Versions - Manage and switch between data versions

- Exercise 5: Import from URL - Import external data

Exercise 0: Setup

Make sure DVC is installed: `dvc --version` Does this command run and show a version number above 3.something? Great!

Initialization

Exercise 1: Initialise DVC in repository

Next we want to start using DVC with one of our projects. Let's follow DVC's documentation on getting started:

1. Run `dvc init`

   ### Getting an *ERROR: failed to initiate DVC - path is not tracked by any supported SCM tool (e.g. Git)* error?

   Make sure you have initialised git in your directory. Run the following to start a git repo

   ```
   git init
   ```

   If this does not work, try and run it with the --subdir argument if you are initialising this in a subdirectory of your project.

   ```
   dvc init --subdir
   ```

2. Check which files DVC added to the repo (*what is each file used for?*)

Hint

```
git status
```

1. .dvc/.gitignore (dvc-specific things for git to ignore, like local config.local)
2. .dvc/config (project-level dvc config, keeps tracks of various settings like remotes, local path to auth, etc)
3. .dvcignore (file types for dvc to ignore if e.g. adding directories)

Git commit new files

Hint

```
git commit -m "YOUR COMMIT MSG"
```

Tracking

Exercise 2: Start tracking files

It's time to add files that we want to track with DVC. We don't want to include the files in our repo but instead track and use references to the data stored elsewhere.

There's multiple storage solutions in DVC. Generally they require some kind of authentication so that you can read and write from the location but that is beyond the scope for now. Instead we will just use a "local" remote -- that is, just a different directory on your laptop than this project and pretend it's a "remote" remote.

1. Fetch data locally:

```
mkdir -p data/raw
wget https://raw.githubusercontent.com/Jeppe-T-K/itu-data-science-in-productio
```

2. Add file to tracking with DVC

Hint: It's essentially the same as git

```
dvc add data
```

3. Ensure metadata files are tracked by git (*which files are added?*)

Hint: add and commit newly created files

```
git add data.dvc .gitignore
git commit -m "Added data/ to dvc"
```

4. What info does each .dvc file contain?

Hint: Open the files in a text editor or on unix based systems (e.g. Linux or Mac) you can use `less` or `cat` in the terminal to view them

less (view individual files):

```
    less *.dvc # use :n, :p, q, to go to next file, previous file and
```

cat (prints the contents of all files):

```
    cat *.dvc
```

note: The * here just means anything so in this case any file name with the .dvc extension

Exercise 3: Using a remote

Remotes work in a similar way as with git: it's some place else that keeps track of you data. DVC stores the data in an efficient binary format and allows you to fetch the data again easily and quickly when needed for e.g. ML stuff.

> ***Note**: we will simply download the data locally and work with it there, but if you have a remote set up already with data, you can follow this method for using "external data" (not necessary for now).*

1. Configure remote ("local") with `dvc remote add -d <remote_name> /localpath/to/remote`

   ### Want to use actual remote?

   You're welcome to try and set it up. Take your pick from all these options

2. Add and commit changes to .dvc/config

   ### Hint: you know the flow now with git

   ```
   git add .dvc/config
   git commit -m "Added remote to dvc"
   ```

3. Push DVC data to remote

   ### Hint: It's essentially the same as git

   ```
   dvc push
   ```

4. You accidentally (or intentionally) deleted your local data in /data/raw. How

would you fetch it again?

Hint: It's essentially the same as git

```
dvc pull
```

Versions

Exercise 4: Switching between data versions

Quite often you don't use the raw data directly in your models but run it through a data pipeline to create a cleansed or ML-ready dataset. That also means that the files/artifacts you create and want to keep track of can change with every run of your code.

For this exercise, we will essentially follow the steps in Exercise 1, but in addition we will switch to another branch in git and make changes to our cleansed data. The challenge is then to recover the original data on the main branch!

1. Create a "cleansed" version of your data (or use this pre-made)

```
mkdir -p data/cleansed
wget https://raw.githubusercontent.com/Jeppe-T-K/itu-data-science-in-productio
```

2. Add new data to dvc + git

Hint: repeat steps 2-3 from Exercise 1

```
dvc add data/
git add data.dvc
git commit -m "Added cleansed data"
(git push)
dvc push
```

3. Create new branch in git

Hint

```
git checkout -b "my_branch_name"
```

4. Modify your cleansed data (*save to same file name!*)

5. Add changed file to dvc + git

Hint: repeat steps from step 2

```
dvc add data/
git add data.dvc
git commit -m "Added cleansed data from new method"
(git push)
dvc push
```

6. To back to the main branch and check out your cleansed data (*is it the same that you originally added to dvc?*)

Hint

No, it's not. Not unless you made a mistake, anyway.

To go back to the main branch, you can run `git checkout main`

```
dvc pull
```

Import

Exercise 5: Import from URL

Sometimes you rely on external data sources, for example if a remote system updates the data file or if DVC is not used by that system. In order to do that, we can import directly from a url:

1. Import the data

   ### Hint: Use the import-url command

   ```
   mkdir data_imported
   dvc import-url https://raw.githubusercontent.com/Jeppe-T-K/itu_sdse_2
   git add data_imported/*
   git commit -m "Added cleansed data"
   (git push)
   dvc push
   ```

2. How can you avoid downloading data directly? And why would you want that?

   ### There are two commands

   1. --no-download
   2. --to-remote

--no-download simply creates the DVC file.

--to-remote also transfers the file to your DVC remote.

3. How do you make sure the tracked data is updated?

You need to *update* via DVC

dvc update --to-remote data_imported/coco_edited_highres.jpeg

Outro

And that's it! Now you've done the whole flow of tracking raw data, creating + modifying a cleansed dataset, and switching between different data versions.

Much of the same thinking about version control of big files also applies to models. However, the flow for creating, testing, evaluating and deploying models through *experiments* usually require a bit more infrastructure, so before going crazy with exercises for that, we'll instead focus next time on creating a project that follows a nice, clean structure to make our lives easier.